

Volatility Forecasting: A Statistical and Deep Learning Approach

Abstract

1 Introduction

1.1 Context

Modern share trading is often traced to Amsterdam in 1602, when the Dutch East India Company issued transferable shares and investors began trading them with one another [1]. At that point in time, it originated the possibility to earn money by buying the paper at a lower price, hoping that the company would grow with its expansion. Or, if someone had privileged information, e.g. a failed expedition or a successful one, would be in a better position than the others, being able to sell its position before it loses value or buy more before it gets a higher price.

The desire to obtain and interpret information that may affect stock prices remains central to financial markets. Services such as the Bloomberg Terminal give investors rapid access to news and data [2], while some traders use direct exchange feeds when even small delays matter [3]. If you receive the information before every player, you have a leverage over them and can position yourself in a beneficial way. Jim Simons, the founder of Renaissance Technologies, whose fund Medallion had an average annual return of 66% before investor fees, started his trading using news in early human trading decisions, before passing to models who studies historical market data as stated by Gregory Zuckerman in his Jim Simons biography, *The Man Who Solved the Market* [4].

As competition for information increases, the opportunity to profit from that same information decreases. Fama's efficient-market hypothesis states that prices reflect available information; in its weak form, this includes information in past prices [5]. An efficient market is one where prices adjust

quickly enough to available information such that an investor cannot reliably earn extra returns from it after accounting for risk and trading costs. Historical prices can still be studied for candidate patterns, but market efficiency gives no reason to assume that those patterns will yield profits. As described by Zuckerman, Simons searched historical price data for recurring patterns [4].

A historical data is known as a time series. Time series analysis has been a widely explored field in statistics and ML (Machine Learning). Statistical analysis is focused on understanding the properties of the data (stationarity, mean reversion, seasonality, trend, cycles, etc.) and creating a model, based on such characteristics that successfully explains the movements observed in such data. ML, in the meanwhile, is interested in creating a model that efficiently explains the data without effectively needing its traits.

1.1.1 Stastical Models

An early landmark of time series statistical modelling is Yule’s 1927 autoregressive analysis, where the current value depends on past values [6]. In 1927, Slutsky demonstrated that summing random values can produce smooth, apparently cyclical movements [7], and in 1938, Wold showed that a weakly stationary time series can be split into a predictable component and another represented by current and past shocks [8]. Weak stationarity means a constant mean and finite variance, with covariance depending only on time lag. Both contributions served as foundations for moving-average representations.

Box and Jenkins’s 1970 treatment popularized ARIMA (Autoregressive Integrated Moving Average) modeling, combining AR and MA terms with differencing [9]. In ARIMA(p, d, q), p and q correspond to the orders of AR and MA, respectively, while d is the number of differences used to address some forms of non-stationarity, especially stochastic trends. Their framework also treats seasonal ARIMA models. SARIMA(p, d, q)(P, D, Q) s uses (p, d, q) for the nonseasonal component. Seasonality is a pattern that tends to recur every s observations; for monthly data with a yearly pattern, $s = 12$. P and Q are the orders of seasonal AR and MA parts, respectively, and D is the number of seasonal differences.

Box and Jenkins’s models address the series’ expected value over time. In a homoscedastic specification, the unexplained component is modeled as white noise ($\varepsilon_t \sim \text{WN}(0, \sigma^2)$) with constant variance. Robert Engle’s 1982 ARCH (Autoregressive Conditional Heteroskedasticity) model instead studies time-varying conditional variance, originally in the context of uncertainty about future U.K. inflation [12]. A shock can have constant unconditional

variance while its variance conditional on past information changes over time. ARCH captures this by relating current conditional variance to past squared shocks.

Volatility often stays elevated for many periods, so ARCH can require many past squared shocks and consequently many parameters. Volatility is the standard deviation of returns, equal to the square root of their variance. Tim Bollerslev’s 1986 GARCH (Generalized Autoregressive Conditional Heteroskedasticity) model added lagged conditional variance terms to represent this persistence parsimoniously [13].

In 2009, Fulvio Corsi presented the HAR (Heterogeneous Autoregressive) model. Market participants act over different time horizons, so realized volatility measured over a day, a week, and a month may help predict future realized volatility [14]. Corsi drew on the heterogeneous-market hypothesis discussed by Ulrich A. Müller et al. in their 1993 working paper *Fractals and Intrinsic Time: A Challenge to Econometricians* [15].

AR, MA, ARIMA and SARIMA models the conditional mean. HAR presents a Realized-volatility regression. ARCH and GARCH the conditional variance. All shocks were modeled as WN. However, there are models who assume that shocks behaves like a Brownian Motion, mostly in econometry.

In 1900, Bachelier modeled speculative price changes using a Brownian-motion precursor [16]. In 1959, Osborne studied Brownian behavior in log price changes [17]. Samuelson’s 1965 warrant-pricing model used lognormal asset prices, corresponding to geometric Brownian motion [18]. In 1973, Fischer Black and Myron Scholes derived the theoretical price of a European call option on a non-dividend-paying stock under a geometric-Brownian-motion price model [19]. In 1998, Fabienne Comte and Eric Renault presented a continuous-time stochastic-volatility model with long memory [20]. Gatheral, Jaisson, and Rosenbaum’s 2014 preprint, published in 2018, proposed rough fractional stochastic volatility with a Hurst exponent below one half [21].

1.1.2 ML Models

In Machine Learning, the main idea is to create a model able to learn patterns from data. Neural networks combine weighted inputs through nonlinear layers, and their parameters can be adjusted by backpropagation. Werbos described the method in his 1974 dissertation [22]; Rumelhart, Hinton, and Williams demonstrated its use for learning internal representations in multi-layer networks in 1986 [23].

In 1987, Lapedes and Farber studied feedforward neural networks for nonlinear signal prediction and system modeling [28]. Waibel et al. used a time-delay neural network (TDNN) for phoneme recognition in 1989, apply-

ing shared computations across successive time positions [29]. Bai, Kolter, and Koltun evaluated a generic temporal convolutional network (TCN) with causal, dilated convolutions in 2018 [30]. Reisenhofer et al. developed HAR-Net in 2022, using causal convolutional layers that can be initialized to reproduce a fitted HAR forecast before optimization [31].

Elman’s 1990 recurrent neural network illustrates how a model can process sequential observations while carrying a hidden state forward [32]. Standard RNNs can have difficulty learning long-range dependencies because gradients may vanish or explode. Hochreiter and Schmidhuber introduced the LSTM (Long Short-Term Memory) in 1997 to help preserve error signals over longer spans [33]. Cho et al. introduced the gated recurrent unit (GRU) in 2014, using a simpler gated state than the LSTM; its parameter count and training speed depend on the chosen architecture and implementation [34].

In 2001, Breiman presented random forests, which aggregate predictions from randomized decision trees; for regression, their outputs are averaged [35]. Chen and Guestrin introduced XGBoost in 2016 as a scalable implementation of gradient-boosted trees, fitted sequentially to improve an objective function [36].

Vaswani et al. introduced the Transformer in 2017. Its self-attention relates positions in a sequence without a recurrent pass; autoregressive forecasting additionally requires a causal attention mask [37].

Each of these models have been used to predict returns or volatility even if they are not explicitly built for that reason as a HAR model is, for example, and that’s the main reason they are useful. They are not limited to one scope. One of the main benefits of ML models is generality.

1.1.3 Estimating Volatility

Unlike prices and returns, volatility is latent. It describes the dispersion of possible returns, so a single observed return cannot reveal its value: a single return reflects both the prevailing volatility and the particular random shock that occurred. In a continuous-time model, instantaneous volatility may also change within a trading day. Consequently, neither a daily closing price nor one daily return identifies the variance accumulated over that day.

Daily variance must therefore be estimated from observable prices. With sufficiently frequent observations and under suitable assumptions, the sum of squared intraday log returns approximates the day’s integrated variance; this is known as realized variance [38]. When intraday data are unavailable, range-based estimators use information in daily prices: Parkinson uses the high and low [40], while Garman–Klass uses the open, high, low, and close [41]. All are estimates rather than exact observations, and their accuracy

depends on the available data and the assumptions behind each estimator.

1.2 Motivations

The main objective with this project is understanding the better approach for predicting the price of an asset. The potential capabilities of ML models in finance forecasting tasks are tremendous, although the stability and robustness of statistical models are extremely reliable.

Several approaches have been presented as comparing both types of models. However, our objective is to extend these comparisons to include, besides the baseline models (AR, SARIMA, GARCH, HAR, MLP), HARnet and RFSV models.

Furthermore, we are interested in understanding the generalization capacity of such models. Our focus is to understand the difference in performance between a local (fitted by one equity data) and a global (fitted by several tickers) model.

1.3 Contributions

In this paper, we present the comparison between AR(1), SARIMA(1,0,1)(1,0,1,5), GARCH(1,1), HAR-20 (base HAR using averages from 1, 5 and 20 observations), HAR-80 (HAR-20 extended to 40 and 80 observation averages), RFSV, LSTM, SiLU-LSTM (tanh activation function changed by SiLU), HARnet-20, HARnet-80. Each model will be fitted locally, one model fitted per each of 5 top-volume tickers, and globally, across a dataset of 2.714 tickers from NYSE and NASDAQ currency exchanges.

1.4 Related Work

Corsi’s heterogeneous autoregressive (HAR) model uses averages of realized volatility over daily, weekly, and monthly horizons to approximate persistent volatility dynamics with a small number of coefficients. Its empirical fits use intraday data for the USD/CHF spot exchange rate (Reuters quotes, 1989–2003), S&P 500 index futures (1990–2007), and 30-year U.S. Treasury bond futures (1990–2003); the equity exposure is an index future, not individual stocks [14]. Audrino and Chassot compare HAR with machine-learning methods using five-minute returns from NYSE TAQ trades in 2015–2023 for 1,445 U.S. stocks matched to CRSP. Their full sample includes 27 Dow Jones Industrial Average constituents and 69 Nasdaq-100 constituents, but is not restricted to those indexes. They show that the HAR fitting window

and re-estimation frequency materially affect the comparison; in their setting, a carefully fitted HAR remains a strong benchmark [42]. Reisenhofer, Bayer, and Hautsch extend this baseline with HARNet, a hierarchy of dilated causal convolutions initialized to reproduce a fitted HAR forecast before further optimization. They fit and test both 20- and 80-observation variants on five-minute realized-variance series for the S&P 500, FTSE, and Dow Jones Industrial Average indexes from the Oxford-Man Institute’s Realized Library, covering 2001–2020. These are index series rather than individual equities [31].

The choice between fitting one model per asset and pooling assets is another central question. Montero-Manso and Hyndman define a *local* method as fitting a separate forecasting function to each series and a *global* method as fitting one function using observations pooled across series. They show that a sufficiently flexible global method can represent the same forecasts as local methods even when the series differ. With a fixed finite input window, however, one global function must give the same forecast to identical inputs, whereas two local functions can differ; longer histories or series identifiers can distinguish such cases. Pooling increases the number of training examples available for shared parameters, but this is a modeling opportunity, not a guarantee that a global fit will forecast better. Their experiments use non-equity benchmarks including M1–M4 competition series, ATM cash demand, Wikipedia traffic, weather, and electricity demand, so their results motivate the local-versus-global design rather than directly establish its performance for stocks [43]. Liu et al. use 2014–2023 daily closing prices from Reuters Refinitiv Workspace for 11,771 individual stocks from exchanges in Asia, Europe, North America, and elsewhere; 10,240 series are available for global training and 1,531 stocks are reserved for unseen-stock evaluation. They report benefits from larger and more heterogeneous training pools [44]. Zhang et al. use Nasdaq ITCH order-book data supplied by LOBSTER for 93 S&P 500 constituent stocks selected from the 100 most liquid, spanning July 2011 to June 2021. They compare stock-specific and pooled models for intraday realized volatility, add a market-volatility proxy, and test previously unseen stocks [45]. These studies motivate evaluating local and pooled models separately, while recognizing that intraday predictors are not available for every asset in the present equity dataset.

Rough-volatility studies provide a complementary statistical approach. Gatheral, Jaisson, and Rosenbaum estimate roughness from DAX equity-index and German Bund bond futures using QuantHouse intraday data (2010–2014), and from S&P and Nasdaq index realized-variance series in the Oxford-Man library (2000–2014). They find evidence that log-volatility is rough and propose a rough fractional stochastic-volatility model with a

Hurst exponent below one half; this is estimation and model calibration, not neural-network training [21]. Mouti investigates roughness using Bloomberg daily open, high, low, and close data from April 2005 to April 2015. The asset set includes U.S. and international equity indexes (S&P 100, S&P 400, IBEX 35, FTSE 100, Hang Seng, and others) and individual shares such as Google and Microsoft; the main illustrations use S&P 100 and IBEX 35. Mouti also compares rough-volatility forecasts with AR, HAR, and GARCH forecasts [46]. Tang, Rosenbaum, and Zhou compare a pooled LSTM with a parsimonious rough-volatility model using five-minute Binance prices for 213 cryptocurrencies quoted against USDT. Their training period is 2020–2021 and their test period covers the first half of 2022; no equities or equity indexes are in that sample [47].

Finally, Brini compares pretrained, zero-shot time-series foundation models with econometric realized-volatility benchmarks on the VOLARE evaluation set: 40 individual U.S. stocks from all 11 GICS sectors (2015–2026), five major foreign-exchange pairs, and five commodity or equity-index futures. VOLARE’s five-minute realized measures are derived from Kibot tick data. The 50 assets are used for evaluation and for rolling estimation of the econometric benchmarks, not for task-specific training of the zero-shot foundation models. Brini finds that conclusions depend on how errors are aggregated across assets and whether forecasts are recalibrated [48]. The present thesis investigates models fitted on its own daily variance proxies, with local and global equity fits and matched forecast observations as the basis for eventual comparison. No result from the cited papers establishes the ranking of those models on this dataset.

1.5 Outline

On Section 2, we’ll be explaining the concepts behind volatility prediction, starting from statistical theory until ML theory necessary to understand the paper. On Section 3, we’ll be describing each model and our framework for evaluation. On Section 4, we’ll focus on implementation and reproducibility. On Section 5, we present our results. On Section 6, conclusion and further work.

2 Background

2.1 Statistics Concepts

2.1.1 Moments

Let $Y(t)$ be a random variable observed over time, and let $\mathcal{F}(t-1)$ contain the information available before time t . An unconditional property describes the distribution of $Y(t)$ without using that history; a conditional property describes its distribution given $\mathcal{F}(t-1)$. The distinction for the mean is

$$\mu = E[Y(t)], \quad \mu(t) = E[Y(t) \mid \mathcal{F}(t-1)]. \quad (2.1)$$

A moment summarizes an aspect of a distribution: its first moment concerns location, its second central moment dispersion, and its third central moment asymmetry.

For n observations, the sample mean estimates the population mean:

$$\bar{y} = \frac{1}{n} \sum_{t=1}^n y(t). \quad (2.2)$$

An autoregressive model illustrates a conditional mean. With the model shock $\varepsilon(t)$ satisfying $E[\varepsilon(t) \mid \mathcal{F}(t-1)] = 0$,

$$Y(t) = c + \phi Y(t-1) + \varepsilon(t), \quad \mu(t) = c + \phi Y(t-1). \quad (2.3)$$

SARIMA extends autoregressive and moving-average modeling with ordinary and seasonal differencing; it likewise specifies a forecast of the conditional mean of the modeled series. Differencing must be reversed when expressing a forecast on the original scale [9]. A fitted residual is the observed value minus the fitted conditional mean,

$$r(t) = y(t) - \hat{\mu}(t). \quad (2.4)$$

Thus $r(t)$ is calculated after fitting from observed data, whereas $\varepsilon(t)$ denotes the model's unobserved error or shock; fitted residuals approximate shocks.

The population variance and its sample estimate are

$$\text{Var}(Y(t)) = E[(Y(t) - \mu)^2], \quad s^2 = \frac{1}{n-1} \sum_{t=1}^n (y(t) - \bar{y})^2. \quad (2.5)$$

The estimator s^2 is unbiased for independent, identically distributed observations with finite variance: across repeated samples, its expected value equals

the population variance. Dividing by $n - 1$ compensates for estimating μ with the same sample's mean; dividing by n would underestimate variance on average. With dependent time-series observations, s^2 remains descriptive, but that unbiasedness is not automatic.

Conditional variance describes uncertainty left after observing the past. For $\varepsilon(t) = Y(t) - \mu(t)$, its conditional mean is zero, so

$$h(t) = \text{Var}(Y(t) \mid \mathcal{F}(t-1)) = \text{E}[\varepsilon(t)^2 \mid \mathcal{F}(t-1)]. \quad (2.6)$$

ARCH models $h(t)$ from past squared shocks; GARCH also uses past conditional variance:

$$h(t) = \omega + \alpha \varepsilon(t-1)^2 \quad \text{ARCH}(1), \quad (2.7)$$

$$h(t) = \omega + \alpha \varepsilon(t-1)^2 + \beta h(t-1) \quad \text{GARCH}(1,1). \quad (2.8)$$

In estimation, lagged fitted residuals $r(t-1)$ stand in for the unobserved shocks. With $\omega > 0$, $\alpha, \beta \geq 0$, and $\alpha + \beta < 1$ (taking $\beta = 0$ for ARCH), these models can have constant unconditional variance while conditional uncertainty changes [12, 13].

Covariance generalizes variance to two variables:

$$\text{Cov}(X, Y) = \text{E}[(X - \text{E}[X])(Y - \text{E}[Y])], \quad \text{Var}(X) = \text{Cov}(X, X). \quad (2.9)$$

Applied to $Y(t)$ and $Y(t-k)$, it measures linear dependence across a lag k . The standardized third central moment is skewness, when the variance is positive and the third moment exists:

$$\text{Skew}(Y(t)) = \frac{\text{E}[(Y(t) - \mu)^3]}{\text{Var}(Y(t))^{3/2}}. \quad (2.10)$$

Its sign indicates the direction of asymmetry.

Homoscedasticity means that error variance is constant across observations; conditional homoscedasticity specifically means the variance given past information stays fixed. Heteroscedasticity means it changes. Thus a homoscedastic model has

$$\text{Var}(\varepsilon(t) \mid \mathcal{F}(t-1)) = \sigma^2, \quad (2.11)$$

whereas ARCH and GARCH set this conditional variance to $h(t)$ from (2.7) or (2.8). Their changing conditional variance does not itself imply changing unconditional variance [12, 13].

2.1.2 Shock Dynamics

A shock is the part of an observation that the conditional-mean model does not predict. Zero-mean white noise has constant finite unconditional variance and zero covariance between distinct dates. White noise need not be Gaussian or independent. In a homoscedastic SARIMA model, the innovations are commonly modeled as white noise with constant conditional variance. In ARCH and GARCH, the shocks follow

$$\varepsilon(t) = \sqrt{h(t)}z(t), \quad (2.12)$$

where the standardized innovations $z(t)$ are commonly assumed independent with mean zero and variance one. The shocks can be uncorrelated with constant unconditional variance while $h(t)$ changes [9, 12, 13].

In continuous time, standard Brownian motion $W(t)$ starts at zero, has continuous sample paths, has independent increments, and has stationary Gaussian increments:

$$W(0) = 0, \quad W(t) - W(s) \sim N(0, t - s), \quad t > s. \quad (2.13)$$

The increment variance is Δt , while its standard deviation is $(\Delta t)^{1/2}$. Although its increments are stationary, $W(t)$ itself is not:

$$\text{Var}(W(t)) = t. \quad (2.14)$$

White noise describes uncorrelated values at distinct dates; Brownian motion is a continuous-path process whose increments over disjoint intervals are independent.

Arithmetic Brownian motion adds drift and scale to that driver:

$$S(t) = S(0) + \mu t + \sigma W(t). \quad (2.15)$$

Its changes can be positive or negative without keeping $S(t)$ positive. Geometric Brownian motion instead models proportional changes:

$$dS(t) = \mu S(t) dt + \sigma S(t) dW(t). \quad (2.16)$$

To solve this equation, consider a general Brownian diffusion

$$dX(t) = a(t) dt + b(t) dW(t). \quad (2.17)$$

For a twice continuously differentiable, time-independent function f , the one-dimensional form of Itô's lemma states

$$\begin{aligned} df(X(t)) &= \left[a(t)f'(X(t)) + \frac{1}{2}b(t)^2 f''(X(t)) \right] dt \\ &\quad + b(t)f'(X(t)) dW(t). \end{aligned} \quad (2.18)$$

Because $W(0) = 0$, a Brownian increment of length Δt has the exact scaling in distribution

$$W(\Delta t) \stackrel{d}{=} (\Delta t)^{1/2} Z, \quad Z \sim N(0, 1). \quad (2.19)$$

Thus its typical size is approximately $(\Delta t)^{1/2}$, and its square contributes at order Δt . Over uniform partitions of $[0, t]$, the squared increments have the nonzero limit

$$\sum_{i=0}^{n-1} \left[W\left(\frac{(i+1)t}{n}\right) - W\left(\frac{it}{n}\right) \right]^2 \xrightarrow[n \rightarrow \infty]{P} t. \quad (2.20)$$

This quadratic variation produces the second-derivative term in Itô's lemma. For $f(x) = \log x$ on $x > 0$,

$$f'(x) = \frac{1}{x}, \quad f''(x) = -\frac{1}{x^2}. \quad (2.21)$$

With $X(t) = S(t)$, $a(t) = \mu S(t)$ and $b(t) = \sigma S(t)$, Itô's lemma gives the extra $-\sigma^2/2$ term that an ordinary chain rule would miss:

$$d \log S(t) = (\mu - \sigma^2/2) dt + \sigma dW(t). \quad (2.22)$$

Integrating from zero to t , using $W(0) = 0$, gives

$$\log \frac{S(t)}{S(0)} = (\mu - \sigma^2/2)t + \sigma W(t). \quad (2.23)$$

Exponentiating gives a positive level for $S(0) > 0$, making geometric Brownian motion suitable for positive-valued quantities [16, 18, 19]:

$$S(t) = S(0) \exp\{(\mu - \sigma^2/2)t + \sigma W(t)\}. \quad (2.24)$$

Fractional Brownian motion $B^H(t)$ replaces the Brownian scaling exponent $1/2$ with a Hurst parameter $H \in (0, 1)$. Its centered Gaussian increments are stationary and satisfy

$$\text{Var}(B^H(t + \Delta t) - B^H(t)) = (\Delta t)^{2H}, \quad (2.25)$$

$$\text{SD}(B^H(t + \Delta t) - B^H(t)) = (\Delta t)^H. \quad (2.26)$$

The case $H = 1/2$ recovers standard Brownian motion. For $H > 1/2$, fractional Brownian increments are positively correlated across time, indicating persistence and long memory in the increment process. For $H < 1/2$, the paths are rougher than Brownian paths: at increasingly fine time scales they remain continuous but look more jagged, because typical changes shrink more slowly as the time interval decreases [21].

Fractional Ornstein–Uhlenbeck (fOU) log volatility combines a mean-reverting drift with fractional Brownian forcing:

$$dX(t) = -\alpha(X(t) - m) dt + \nu dB^H(t), \quad \alpha > 0. \quad (2.27)$$

Its stationary form is

$$X(t) = m + \nu \int_{-\infty}^t e^{-\alpha(t-s)} dB^H(s). \quad (2.28)$$

Deviations from m decay at a rate set by α , while H controls local roughness [21].

2.1.3 Parameter estimation

Statistical fitting chooses model parameters from training observations by optimizing a specified criterion. For a linear conditional-mean model $y(t) = x(t)^\top \beta + \varepsilon(t)$, ordinary least squares (OLS) minimizes squared residuals:

$$\hat{\beta}_{\text{OLS}} \in \arg \min_{\beta} \sum_{t \in \mathcal{T}_{\text{train}}} (y(t) - x(t)^\top \beta)^2. \quad (2.29)$$

Here β is the vector of model parameters, and $x(t)$ contains the regressors available for the forecast, including an intercept if required. OLS fits the conditional mean of models such as AR and HAR; Gaussian errors are not needed to compute it. When errors are also independent Gaussian, OLS maximizes their likelihood conditional on the regressors.

Maximum likelihood instead chooses parameters that make the observed training series most probable under the assumed statistical model. With a conditional Gaussian mean $\mu_\theta(t)$ and positive conditional variance $\sigma_\theta^2(t)$, its log likelihood, up to initialization terms, is

$$\log L(\theta) = -\frac{1}{2} \sum_{t \in \mathcal{T}_{\text{train}}} \left[\log(2\pi\sigma_\theta^2(t)) + \frac{(y(t) - \mu_\theta(t))^2}{\sigma_\theta^2(t)} \right], \quad \theta^* \in \arg \max_{\theta} \log L(\theta). \quad (2.30)$$

The negative log likelihood defines a loss for a minimizing optimizer:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\log L(\theta), \quad \theta^* \in \arg \min_{\theta} \mathcal{L}_{\text{NLL}}(\theta). \quad (2.31)$$

The two objectives give the same θ^* : minimizing the negative log likelihood finds the parameters that best fit the observations under the assumed distribution. Unlike OLS with constant error variance, this criterion accounts for a variance that changes with the past, as in GARCH [13].

Rough fractional stochastic volatility (RFSV) instead estimates roughness from moments of log-volatility increments. Following Gatheral, Jaisson, and Rosenbaum's Section 2.1 notation, observe positive volatility on a grid $\sigma_0, \sigma_\Delta, \dots, \sigma_{N\Delta}$ over $[0, T]$, where $N = \lfloor T/\Delta \rfloor$. For moment order $q \geq 0$, their empirical moment is

$$m(q, \Delta) = \frac{1}{N} \sum_{k=1}^N |\log \sigma_{k\Delta} - \log \sigma_{(k-1)\Delta}|^q. \quad (2.32)$$

To interpret its slope and intercept, write W^H for the fractional Brownian driver B^H introduced above. Fractional Brownian increments obey

$$\mathbb{E}[|W^H(t + \Delta) - W^H(t)|^q] = K_q \Delta^{qH}, \quad K_q = \mathbb{E}[|Z|^q], \quad Z \sim \mathcal{N}(0, 1). \quad (2.33)$$

Thus K_q is the q th absolute moment of a standard Gaussian variable. Let $X(t) = \log \sigma(t)$. When the fOU mean-reversion rate is small over the lags considered, its increments approximately inherit the scaled fractional Brownian moments [21]:

$$\mathbb{E}[|X(t + \Delta) - X(t)|^q] \approx \nu^q K_q \Delta^{qH}, \quad m(q, \Delta) \approx \nu^q K_q \Delta^{qH}. \quad (2.34)$$

The second approximation treats the empirical moment as an estimate of the theoretical one. Taking logarithms for $q > 0$ and positive moments gives

$$\log m(q, \Delta) \approx q \log \nu + \log K_q + qH \log \Delta. \quad (2.35)$$

Consequently, in a regression across lags,

$$\log m(q, \Delta) \approx a_q + \zeta_q \log \Delta, \quad \zeta_q = qH, \quad a_q = \log(\nu^q K_q). \quad (2.36)$$

The slope ζ_q identifies roughness H , while the intercept a_q contains the increment scale together with K_q . This Gaussian-moment factor does not disappear for general q . Specifically, $K_2 = \mathbb{E}[Z^2] = 1$, so

$$a_2 = \log(\nu^2), \quad \nu^2 = e^{a_2}. \quad (2.37)$$

The fitted $q = 2$ intercept therefore estimates ν^2 as e^{a_2} , whereas the family of fitted slopes estimates H from the relation $\zeta_q = qH$ [21]. The purpose of this scale estimate appears in the paper's Section 5.2 variance predictor [21]:

$$\widehat{\sigma}_{t+\Delta}^2 = \exp\left\{\widehat{\log \sigma_{t+\Delta}^2} + 2c(H)\nu^2 \Delta^{2H}\right\}. \quad (2.38)$$

where

$$c(H) = \frac{\Gamma(3/2 - H)}{\Gamma(H + 1/2)\Gamma(2 - 2H)}. \quad (2.39)$$

Here $\widehat{\log \sigma_{t+\Delta}^2}$ is the conditional log-variance forecast from Section 5.1, and the $2c(H)\nu^2\Delta^{2H}$ term adjusts its exponent for conditional variance [21]. Thus H controls how increment size scales with lag and ν^2 its scale. The moment estimates use a volatility proxy because spot volatility is not directly observed; the proxy, sampling interval, and chosen lags can affect the estimates.

2.2 ML Concepts

2.2.1 Neural networks and layers

A neural network is a parameterized composition of layers that maps inputs to predictions. Let $x^{(0)} = x$ be its input vector. In a feedforward layer,

$$z^{(\ell)} = W^{(\ell)}x^{(\ell-1)} + b^{(\ell)}, \quad x^{(\ell)} = \phi(z^{(\ell)}). \quad (2.40)$$

The weights $W^{(\ell)}$ and biases $b^{(\ell)}$ are learned; the activation ϕ supplies non-linearity. Without nonlinear activations, a stack of such layers reduces to one affine map. The final layer maps the learned representation to the forecast, with its output transform chosen to match the target's units.

Common elementwise activations, for a scalar z , are

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad (2.41)$$

$$\text{ReLU}(z) = \max(0, z), \quad \text{SiLU}(z) = z \text{sigmoid}(z). \quad (2.42)$$

Sigmoid maps to $(0, 1)$ and is useful for gates; tanh maps to $(-1, 1)$ and gives signed, bounded states. Both saturate at large $|z|$, yielding small derivatives there. ReLU has a zero derivative on negative inputs, so a unit that stays negative may stop learning. SiLU is smooth and retains a small nonzero response for negative inputs.

2.2.2 Losses, backpropagation, and optimization

A loss function $\ell(\hat{y}, y)$ assigns a numerical cost to predicting $\hat{y}$ when the observed value is y . Following the empirical-risk formulation of Reisenhofer, Bayer, and Hautsch [31], for a set $Z = \{(x_i, y_i)\}_{i=1}^N$ and model f_θ with parameters θ , the mean loss is

$$J(\theta; Z) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i). \quad (2.43)$$

For evaluation, write $\hat{y}_i = f_\theta(x_i)$ and $e_i = y_i - \hat{y}_i$. Let $s_i > 0$ be the naive scale associated with observation i . A one-step naive forecast simply repeats

the last observed value, so its mean absolute change over a reference history provides a scale for judging whether errors are large relative to that simple forecast. With these definitions, the five metrics are

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |e_i|, \quad \text{MASE} = \frac{1}{N} \sum_{i=1}^N \frac{|e_i|}{s_i}, \quad (2.44)$$

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N e_i^2, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \quad (2.45)$$

$$\text{QLIKE} = \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{\hat{y}_i} - \log\left(\frac{y_i}{\hat{y}_i}\right) - 1 \right]. \quad (2.46)$$

- **MAE** measures a typical absolute miss in the target’s units and is less sensitive to large errors than squared-error measures.
- **MASE** compares absolute errors with the naive scale and is dimensionless; a value below one means average scaled error below that reference scale, provided the scales are defined and positive [25].
- **MSE** squares errors, emphasizing large misses, and has squared target units.
- **RMSE** keeps the squared-error emphasis but returns to the target’s units.
- **QLIKE** is dimensionless and is zero for an exact prediction. Its logarithm requires $y_i > 0$ and $\hat{y}_i > 0$. For fixed $y_i > 0$, QLIKE diverges rapidly as $\hat{y}_i \rightarrow 0^+$, strongly penalizing underprediction, and grows without bound more slowly as $\hat{y}_i \rightarrow +\infty$. It has no real-valued limit as $\hat{y}_i \rightarrow -\infty$ because negative predictions are outside its domain [26].

An optimizer searches for parameters that minimize the chosen training loss, while evaluation can report several metrics on held-out observations. For training examples Z_{train} , the objective is

$$\theta^* \in \arg \min_{\theta} J(\theta; Z_{\text{train}}) = \arg \min_{\theta} \frac{1}{|Z_{\text{train}}|} \sum_{(x,y) \in Z_{\text{train}}} \ell(f_{\theta}(x), y). \quad (2.47)$$

The argmin is the set of parameter values attaining the smallest training loss; an optimizer aims to approach one of them. Backpropagation applies

the chain rule from the loss through each layer to compute $\nabla_{\theta} J$ [23]. Gradient descent then updates the parameters by

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k; Z_k). \quad (2.48)$$

Here Z_k is the training batch for update k , and the learning rate $\eta > 0$ controls the size of each step in parameter space. Too large a step can overshoot a low-loss region; too small a step can make fitting slow. An optimizer specifies the update rule: ordinary stochastic gradient descent uses the current batch gradient, while Adam adapts each parameter’s step using running estimates of the gradient’s first and second moments [24]. Neither backpropagation nor an optimizer guarantees a global minimum.

2.2.3 Temporal convolutions

An ordinary one-dimensional convolution slides the same finite filter h along a sequence $x(t)$. If its supported offsets form a finite set S of integers, its output is

$$(x * h)(t) = \sum_{j \in S} h_j x(t - j). \quad (2.49)$$

The *receptive field* is the part of the input sequence that can affect one output. A filter with K consecutive weighted mappings covers K input positions in one layer; stacking layers can extend that field. Here K is the filter size, or the number of input-to-output mappings in the filter. An ordinary centered filter may use future observations. For forecasting, a *causal* convolution restricts these K mappings to the present and past:

$$(x *_c h)(t) = \sum_{j=0}^{K-1} h_j x(t - j). \quad (2.50)$$

A *dilated causal* convolution separates neighboring mappings by $d \geq 1$ observations:

$$(x *_c, d h)(t) = \sum_{j=0}^{K-1} h_j x(t - jd). \quad (2.51)$$

Its K mappings span $1 + (K - 1)d$ positions without adding filter weights, although a single dilated layer uses only those K input positions. Bai et al. implemented causal filters with increasing dilations in their TCN architecture to let a convolutional network cover a long history without requiring an equally long filter [30]. HARNet uses hierarchies of such filters to represent multiple time horizons [31].

2.2.4 Model fitting

Fitting a forecasting model starts with a chronological division into training, validation, and test periods. Training is used for parameter optimization by means of gradient descent. Validation data guide choices such as architecture, learning rate, batch size, and when to stop training; the test period is held back for the final assessment. Using test outcomes to choose a model would bias that assessment. Any transform learned from data must likewise be fitted on training data alone.

These choices are *hyperparameters*: they are selected outside the parameter updates rather than learned as model weights. *Hidden size* counts the units in a hidden representation, an intermediate set of learned features passed to later layers. A wider representation can encode more features but increases model size and computation, with no guarantee of better test performance. Training examples are divided into *batches*, each used for one gradient update. Smaller batches give more frequent updates per pass through a fixed dataset, with noisier gradients; larger batches use more memory and fewer updates per pass. An *epoch* completes one pass through all training batches. The number of epochs controls how many passes are allowed, while validation loss can select a checkpoint or trigger early stopping.

2.3 Volatility Proxies

2.3.1 Latent volatility and integrated variance

Following Reisenhofer, Bayer, and Hautsch [31], let $p(t) = \log P(t)$ be the log price and suppose its continuous diffusion obeys

$$dp(t) = \mu(t) dt + \sigma(t) dW(t), \quad (2.52)$$

where $\mu(t)$ is the drift, $W(t)$ is Brownian motion, and $\sigma(t)$ is the latent spot volatility. Its squared value is instantaneous variance. The variance accumulated over day t is the integrated variance

$$IV(t) = \int_{t-1}^t \sigma^2(s) ds. \quad (2.53)$$

Neither $\sigma(t)$ nor $IV(t)$ is directly observed from a finite set of prices, so empirical work uses volatility proxies. Here IV means *integrated variance*.

2.3.2 Realized variance

Divide day t into N equally spaced intervals. The intraday log return for interval n is

$$r(t, n) = p\left(t - 1 + \frac{n}{N}\right) - p\left(t - 1 + \frac{n-1}{N}\right), \quad n = 1, \dots, N. \quad (2.54)$$

Realized variance (RV) is the sum of their squares:

$$\text{RV}_N(t) = \sum_{n=1}^N r(t, n)^2. \quad (2.55)$$

For the continuous diffusion above, with sufficiently fine, frictionless observations, $\text{RV}_N(t)$ is consistent for $\text{IV}(t)$ as $N \rightarrow \infty$ [39, 31]. Consistency means that, as the number of intraday observations in the same day increases, the estimator converges in probability to that day's integrated variance. Market microstructure noise and jumps can alter this limiting interpretation.

2.3.3 Parkinson estimator

When only daily high $H(t)$ and low $L(t)$ are available, Parkinson's range estimator gives the day's variance proxy

$$\hat{\sigma}_{\text{PK}}^2(t) = \frac{[\log(H(t)/L(t))]^2}{4 \log(2)}. \quad (2.56)$$

For a continuously observed, zero-drift Brownian log-price path with constant spot volatility $\sigma(t)$ over a unit-length day, $\text{IV}(t) = \sigma^2(t)$ and the squared log range satisfies [40]

$$\mathbb{E}[\{\log(H(t)/L(t))\}^2] = 4 \log(2) \sigma^2(t). \quad (2.57)$$

This expectation gives the denominator in (2.56). Discrete trading, drift, and jumps can affect its accuracy.

2.3.4 Garman–Klass estimator

Garman and Klass also use the daily open $O(t)$ and close $C(t)$. Their practical reduced estimator combines the squared log range with the squared open-to-close log return:

$$\hat{\sigma}_{\text{GK}}^2(t) = \frac{1}{2} \left[\log\left(\frac{H(t)}{L(t)}\right) \right]^2 - (2 \log 2 - 1) \left[\log\left(\frac{C(t)}{O(t)}\right) \right]^2. \quad (2.58)$$

Under the same continuous, zero-drift Brownian model, the open-to-close return has second moment $\sigma^2(t)$. Together with (2.57), this gives

$$\mathbb{E}[\{\log(C(t)/O(t))\}^2] = \sigma^2(t) \quad (2.59)$$

$$\mathbb{E}[\hat{\sigma}_{\text{GK}}^2(t)] = \frac{1}{2}[4\log(2)\sigma^2(t)] - (2\log(2) - 1)\sigma^2(t) = \sigma^2(t). \quad (2.60)$$

Garman and Klass selected this practical form by dropping small cross-product terms from their more efficient quadratic estimator [41]. Like Parkinson's proxy, it uses prices within the observed session and can be biased by discrete trading or departures from the assumed price process.

2.4 Local and Global complexity

Following Montero-Manso and Hyndman, let $S = \{X_i \in \mathbb{R}^T : i = 1, \dots, K\}$ contain K observed series, and let each forecast cover H future observations. A local algorithm fits a separate forecasting function f_{θ_i} , with its own parameter vector θ_i , to each series. A global algorithm fits one function g_θ and one shared parameter vector θ across the set. For an observed series X_i , their forecasts are

$$\hat{Y}_i^{\text{local}} = f_{\theta_i}(X_i), \quad \hat{Y}_i^{\text{global}} = g_\theta(X_i), \quad i = 1, \dots, K. \quad (2.61)$$

Both forecasting functions map $\mathbb{R}^T$ to $\mathbb{R}^H$; the distinction is whether their fitted parameters are series-specific or shared [43].

To count fitted scalar parameters, define $P(f_\theta) = \dim(\theta)$. The total counts are

$$P_{\text{local}} = \sum_{i=1}^K P(f_{\theta_i}), \quad P_{\text{global}} = P(g_\theta). \quad (2.62)$$

Thus K local models with p parameters each use Kp fitted parameters, whereas a global model with q shared parameters uses q . The paper's formal complexity measure is $\mathcal{H}_i$, the hypothesis class, the set of all possible functions a model can learn. If $\mathcal{H}_i$ is the class for series i and $\mathcal{J}$ the global class, their complexities are

$$C_{\text{local}} = \prod_{i=1}^K |\mathcal{H}_i|, \quad C_{\text{global}} = |\mathcal{J}|. \quad (2.63)$$

The paper's relation in Section 3.4 places these classes on the same worst-case generalization bound when $C_{\text{local}} = C_{\text{global}}$, assuming equal in-sample error, independent series, equal effective sample sizes, and loss bounded in $[0, 1]$.

Parameter complexity is a proxy for this comparison only if each parameter has the same number B of possible values and distinct parameter choices give distinct functions. In that case, equal class cardinalities give

$$B^{P_{\text{local}}} = B^{P_{\text{global}}} \implies P_{\text{global}} = \sum_{i=1}^K P(f_{\theta_i}). \quad (2.64)$$

The paper illustrates this complexity with approximately $B = 2^{64}$ values per double-precision parameter in an autoregressive model with one parameter per lag [43].

2.5 Model Evaluation

Hansen, Lunde, and Nason's model confidence set (MCS) compares a finite set of candidate models under one chosen loss function, without assuming that any candidate is the true data-generating model. Let $\mathcal{M}_0$ be the initial set, and let $L_i(t) = L(y(t), \hat{y}_i(t))$ be model i 's loss on a common evaluation observation at time t . Define the loss difference and its expected value by

$$d_{ij}(t) = L_i(t) - L_j(t), \quad \mu_{ij} = \mathbb{E}[d_{ij}(t)], \quad i, j \in \mathcal{M}_0. \quad (2.65)$$

A negative μ_{ij} favors model i . For a current candidate set $\mathcal{M} \subseteq \mathcal{M}_0$, Equation (1) of Hansen, Lunde, and Nason gives the equal-performance null; its alternative is that at least one pair differs:

$$H_{0,\mathcal{M}} : \mu_{ij} = 0 \quad \text{for every } i, j \in \mathcal{M}, \quad (2.66)$$

$$H_{1,\mathcal{M}} : \mu_{ij} \neq 0 \quad \text{for at least one } i, j \in \mathcal{M}, \quad (2.67)$$

$$\text{Reject } H_{0,\mathcal{M}} : p_{\mathcal{M}} < \alpha.$$

Here $p_{\mathcal{M}}$ is the bootstrap p -value. The paper leaves its loss-difference and alternative-hypothesis definitions unnumbered in Section 2 [27].

One test and elimination pair proposed in their Section 3.1.2 uses each model's sample excess loss relative to the average model in the current set. With $m = |\mathcal{M}|$ and n common evaluation times, define

$$\bar{d}_{i\cdot} = \frac{1}{mn} \sum_{j \in \mathcal{M}} \sum_{t=1}^n d_{ij}(t), \quad t_{i\cdot} = \frac{\bar{d}_{i\cdot}}{\sqrt{\widehat{\text{Var}}(\bar{d}_{i\cdot})}}, \quad (2.68)$$

$$T_{\max,\mathcal{M}} = \max_{i \in \mathcal{M}} t_{i\cdot}, \quad e_{\max,\mathcal{M}} = \arg \max_{i \in \mathcal{M}} t_{i\cdot}. \quad (2.69)$$

The paper leaves these statistic and elimination-rule definitions unnumbered. A bootstrap estimates the null distribution of $T_{\max,\mathcal{M}}$ and the variance in $t_{i\cdot}$.

while accounting for dependence in the loss differences. Start with $\mathcal{M} = \mathcal{M}_0$; after a rejection, remove $e_{\max, \mathcal{M}}$ and repeat. The first set for which the null is not rejected is the $(1 - \alpha)$ MCS [27].

Under the paper’s assumptions, the procedure asymptotically contains all models with minimum expected loss among $\mathcal{M}_0$ with probability at least $1 - \alpha$. A surviving model is therefore not proved to be equally accurate or good in an absolute sense: limited information can leave inferior models in the set. For the volatility comparison, each MCS must use one variance proxy, forecast horizon, loss definition, and matched set of forecast observations; QLIKE and MSE lead to separate confidence sets [27].

3 Methods

4 Implementation

5 Experimental Results

5.1 Goals

5.2 Setting

5.3 Case Studies

5.4 Correctness

5.5 Computational Evaluation

5.6 Technical Evaluation

6 Conclusions

References

- [1] Euronext. (2015). Gong marks exchange anniversary and exchange history website launch. Euronext Amsterdam.
- [2] Bloomberg Professional Services. (n.d.). Bloomberg Terminal: Real-time data, news, and analytics.
- [3] Nasdaq. (n.d.). Nasdaq TotalView: Direct market-data feed.

- [4] Zuckerman, G. (2019). *The man who solved the market: How Jim Simons launched the quant revolution*. Portfolio.
- [5] Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417.
- [6] Yule, G. U. (1927). On a method of investigating periodicities in disturbed series, with special reference to Wolfer’s sunspot numbers. *Philosophical Transactions of the Royal Society of London. Series A*, 226, 267–298.
- [7] Slutsky, E. (1927). The summation of random causes as a source of cyclic processes. *Problems of Economic Conditions*, 3(1), 34–64. English translation in *Econometrica*, 5(2), 105–146 (1937).
- [8] Wold, H. (1938). *A study in the analysis of stationary time series*. Almqvist & Wiksell.
- [9] Box, G. E. P., and Jenkins, G. M. (1970). *Time series analysis: Forecasting and control*. Holden-Day.
- [10] Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.
- [11] The Pennsylvania State University. (n.d.). Linear regression, Gauss–Markov theorem. *STAT 508: Applied Data Mining and Statistical Learning*, Lesson 4.
- [12] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.
- [13] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- [14] Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196.
- [15] Müller, U. A., Dacorogna, M. M., Davé, R. D., Pictet, O. V., Olsen, R. B., and Ward, J. R. (1993). *Fractals and intrinsic time: A challenge to econometricians*. Olsen & Associates working paper.
- [16] Bachelier, L. (1900). Théorie de la spéculation. *Annales scientifiques de l’École Normale Supérieure*, 17, 21–86.

- [17] Osborne, M. F. M. (1959). Brownian motion in the stock market. *Operations Research*, 7(2), 145–173.
- [18] Samuelson, P. A. (1965). Rational theory of warrant pricing. *Industrial Management Review*, 6(2), 13–31.
- [19] Black, F., and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [20] Comte, F., and Renault, E. (1998). Long memory in continuous-time stochastic volatility models. *Mathematical Finance*, 8(4), 291–323.
- [21] Gatheral, J., Jaisson, T., and Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance*, 18(6), 933–949.
- [22] Werbos, P. J. (1974). *Beyond regression: New tools for prediction and analysis in the behavioral sciences* [Doctoral dissertation, Harvard University].
- [23] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [24] Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- [25] Hyndman, R. J., and Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.
- [26] Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256.
- [27] Hansen, P. R., Lunde, A., and Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497. doi:10.3982/ECTA5771.
- [28] Lapedes, A., and Farber, R. (1987). *Nonlinear signal processing using neural networks: Prediction and system modelling* (Technical Report LA-UR-87-1662). Los Alamos National Laboratory.
- [29] Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., and Lang, K. J. (1989). Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(3), 328–339.
- [30] Bai, S., Kolter, J. Z., and Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271.

- [31] Reisenhofer, R., Bayer, X., and Hautsch, N. (2022). HARNet: A convolutional neural network for realized volatility forecasting. arXiv:2205.07719.
- [32] Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.
- [33] Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [34] Cho, K., van Merriënboer, B., Gülçehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP*, 1724–1734.
- [35] Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
- [36] Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD Conference*, 785–794.
- [37] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [38] Andersen, T. G., Bollerslev, T., Diebold, F. X., and Labys, P. (2003). Modeling and forecasting realized volatility. *Econometrica*, 71(2), 579–625.
- [39] Andersen, T. G., and Bollerslev, T. (1998). Answering the skeptics: Yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
- [40] Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *The Journal of Business*, 53(1), 61–65.
- [41] Garman, M. B., and Klass, M. J. (1980). On the estimation of security price volatilities from historical data. *The Journal of Business*, 53(1), 67–78.
- [42] Audrino, F., and Chassot, J. (2024). HARd to beat: The overlooked impact of rolling windows in the era of machine learning. arXiv:2406.08041.
- [43] Montero-Manso, P., and Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting*, 37(4), 1632–1653.

- [44] Liu, C., Tran, M.-N., Wang, C., Gerlach, R., and Kohn, R. (2023). Global neural networks and the data scaling effect in financial time series forecasting. arXiv:2309.02072.
- [45] Zhang, C., Zhang, Y., Cucuringu, M., and Qian, Z. (2022). Volatility forecasting with machine learning and intraday commonality. arXiv:2202.08962.
- [46] Mouti, S. (2023). Rough volatility: Evidence from range volatility estimators. arXiv:2312.01426.
- [47] Tang, S. H., Rosenbaum, M., and Zhou, C. (2023). Forecasting volatility with machine learning and rough volatility: Example from the crypto-winter. arXiv:2311.04727.
- [48] Brini, A. (2026). Forecasting realized volatility with time series foundation models: A comparison with econometric benchmarks. arXiv:2607.05291.